Magpie — Sprint 15 spec: cloud two-phase AI deep-dive + reference lookup (optional)

Audience: Codex. **Author:** Claude. **Status: ready to implement (live-call validation needs Regan + an API key — see §10). Locks:** D-123 (references D-110/D-113/D-114/D-115/D-116/D-117/D-119/D-121/D-124). **References:** Sprint 13 (the `FieldSuggestion` + per-field Approve/Edit/Reject review this reuses), Sprint 14 (the multi-source search panel Phase 2 feeds), F-09 (never warehouse marketplace data), and the human-in-the-loop AI principle. This is the **optional cloud** sprint — the local floor (13) and pricing evidence (14) already deliver value with no cloud.

---

1. Goal

An **opt-in** cloud AI assistant that does the two things AI is genuinely good and safe at, in **two separate clicks**: **Phase 1 — Identify & fill** (look at the photos, propose what the item is and fill its fields) and **Phase 2 — Price-assist** (sharpen the search terms for the now-identified item) — plus a **reference-image lookup** (links out so the user can eyeball a catalogue shot to confirm an ID). Every AI output is **staged behind the per-field Approve/Edit/Reject review** from Sprint 13; **nothing persists without an explicit human action.** The **AI never produces prices or numbers** (D-121) and **never imports third-party images** (D-124). Reuses the Sprint 13 review spine; the whole feature is dark/disabled until the user configures a key.

2. Scope

IN (v1):

- **Cloud AI adapter** + **encrypted credential** + **disabled-until-configured** state (§3, §4).
- **Phase 1 "Identify & fill"** button → staged field suggestions (§5).
- **Phase 2 "Price-assist"** button → sharpened search terms feeding the Sprint 14 search panel (§6).
- **Reference-image lookup** → view-only links out (§7).
- **Cost controls + usage visibility** (§8).

OUT (explicitly):

- ❌ **Any AI-produced price/number** (D-121) — Phase 2 produces *search terms*, not figures; pricing stays the Sprint 14 evidence layer.
- ❌ **Importing / copying / storing / embedding any third-party image** into an item or listing (D-124) — reference lookup is **links only**.
- ❌ Auto-persist of any suggestion (the per-field sign-off is mandatory); auto-approve of catalogue ID / variety / grade.
- ❌ Bulk / automatic AI runs; any "analyze all inventory" button; background AI processing.
- ❌ Filling valuation / grade / catalogue-ID / acquisition / cost / profit as **authoritative** data (D-115).
- ❌ Sending photos that haven't been **EXIF-stripped**.

3. AI adapter (provider-agnostic)

- `AiResearchAdapter` **port + one real provider implementation** + a **deterministic fake** for CI. **Provider-agnostic** (D-113): the schema and UI must **not** assume a specific vendor. Pick the default provider at implementation time after checking current **multimodal quality / image support / pricing / terms**; make it swappable.
- Store per call (secret-free): **provider name, model id, request metadata, response metadata, token/cost estimate** (for the audit + usage counter).
- **CI must pass with no key and no network** — the **fake** proves the structured-response mapping and all sign-off paths. (Real-call quality is validated separately, §10.)

4. Encrypted credential + disabled state

- `AICredential` — the API key stored **encrypted, reusing the** `EbayCredential` **Fernet pattern**. Never logged, never returned in API responses, never committed.
- **The entire cloud feature is disabled unless a key is configured** — a clear "Connect an AI provider to enable the deep-dive" state, never a broken screen. No key ⇒ Phase 1/2 buttons are inert with an explanatory state.

5. Phase 1 — "Identify & fill"

- A button on item detail. On click: take the item's photos (**EXIF-stripped first**) + the category's field list → one multimodal call → a **structured** response mapped to fields → **staged** `FieldSuggestion` **rows** (`source=ai`), each with **confidence band** (high/medium/low/candidate) + **evidence** + **source basis**.
- **Curated, category-specific scope (D-115), separated by kind:** factual item-specifics (evidence + approval) · copywriting drafts (title + short description, editable) · **candidates** (catalogue ID / variety — **candidates-only, never authoritative**) · condition **observations** (phrased as observations unless the human confirms). **Never** fill valuation / grade / catalogue-ID / acquisition / cost / profit as authoritative.
- All of it flows through the **existing Sprint 13 review** (Approve writes with `ai` provenance · Edit writes the human value · Reject leaves the item unchanged · low-confidence/candidate in the separate "leads" section). **Nothing persists without a click** — test-proven.

6. Phase 2 — "Price-assist"

- A separate button, meaningful **after** identification. The AI's job: **sharpen the search terms** for the identified item (better keywords, variety/grade folded in) and feed them into the **Sprint 14 multi-source search panel** — so the angled sold-searches become more precise.
- **No price numbers. No bands. No "estimated value."** (D-121.) The output is *better searches*, which open as the Sprint 14 view-only links; pricing remains own-sales + captured comps. (If the model returns any number, the UI does not surface it as a price.)

7. Reference-image lookup

- For the identified item, provide **view-only links out** to catalogue / image-search results so the user can **eyeball a reference** to confirm the ID.
- **Never copy, download, store, cache, or embed** any third-party image into the item, a draft, or a listing (D-124). This is the same open-not-store discipline as the sold-searches — links the user opens, nothing ingested.

8. Cost controls + usage (D-117)

- **Manual, per-item trigger only** — no bulk, no "analyze all inventory," no background runs.

- **Show the estimated cost before** a call (where the provider exposes it).
- **Configurable monthly budget cap** (not hardcoded); **a visible usage counter**; **provider/model shown**.
- **Per-call audit record** (`ai.research.*` , secret-free).
- **Graceful state** when key/budget is absent or exceeded — never a broken app.

9. Design

Reuse the **dealer's-ledger** tokens; the deep-dive review reuses the Sprint 13 suggestion panel (familiar). The two phases are two clear buttons; the disabled/connect state is calm and explanatory. Quality floor: responsive, keyboard focus, reduced motion.

10. Acceptance criteria (honest closure)

CI / fake-validated (no key, no network):

- The **fake adapter proves structured-response → `FieldSuggestion` mapping** and the curated field scope.
- **Per-field guarantee re-proven for AI suggestions:** Approve writes (`ai` provenance) · Edit writes the human value · Reject leaves the item unchanged · **nothing persists without a click**.
- **No-key disabled state** is graceful (not broken).
- **EXIF-strip before send** is asserted in the send path.
- **Phase 2 surfaces no number as a price**; **reference lookup stores no third-party image** (assert both).
- **Audit is secret-free**; no key/secret in logs or commits.
- Backend + frontend tests green; **dual-lane Validation green**; rebuild + restart; **desktop & phone screenshots**; additive migrations only; **pre-migration encrypted backup** + restore spot-check.

Real-call validated (needs Regan + a configured key — §11):

- A **single real provider call** run **with Regan present** proves the live identify-&-fill path end to end (real multimodal response → staged suggestions → a human Approve/Edit/Reject), cost shown, usage incremented, audit secret-free. Until a key is configured, this path stays **fake-validated** (honest-closure, same discipline as

the eBay production validation — the plumbing is CI-proven; the live vendor path is proven once, for real, with Regan watching).

11. For Codex / Regan

Reasoning: High. Additive migrations only — no destructive migration. **EXIF-strip every photo before any cloud send. Never** print or commit the API key, `.env`, secrets, OAuth tokens, rclone config, backup passphrase, or encryption keys; the audit/usage records are **secret-free**. The **AI never produces a price** (D-121) and the reference lookup **never ingests a third-party image** (D-124) — assert both. Build everything against the **fake** so CI needs no key and no network. **Regan's step before live validation:** pick a provider (check current multimodal quality / cost / terms), get an **API key**, configure it, and **be present for the one real call** (like the eBay production run). Everything else ships and validates without a key. Evidence as `SPRINT15_EVIDENCE.md`. This is the third of three remaining Phase-5 sprints (14 ✓ → **15** → 16 photo fix-up).